

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КАЛЬКУЛЯТОР ВЕСА НА РАЗНЫХ ПЛАНЕТАХ»
Вариант 4

Студент:

К.П. Бутор

Преподаватель:

О.М. Внук

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить методы работы с интегрированной средой разработки Visual Studio Code на объектно-ориентированном языке Go. Научиться разрабатывать в выбранной среде приложение с графическим интерфейсом на базе фреймворка Fyne. Изучить архитектурный паттерн MVC (Model–View–Controller) в вариантах с активной и пассивной моделью и применить его на практике.

В соответствии с вариантом 4 «Калькулятор веса на разных планетах» решались следующие задачи:

1. Установить и настроить Visual Studio Code и комплект разработчика Go, изучить функциональные возможности среды и средства отладки, продемонстрировать работу отладчика.

2. Создать пустой проект под вариант задания, установить расширения для работы с UML и SysML v2 и подобрать аналоги расширений, приведённых в методических указаниях.

3. Разработать приложение с графическим интерфейсом: главную форму с кнопкой «Ввести данные», отдельное окно ввода веса на Земле и вывод результатов расчёта для планет Солнечной системы.

4. Обеспечить запоминание последних введённых данных и их восстановление при повторном вводе, а также вывод сообщения об ошибке при некорректных данных.

5. Разработать и описать каркас MVC с активной моделью, построить диаграмму классов.

6. Ответить на контрольные вопросы и оформить отчёт согласно СТП БГУИР.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интегрированная среда разработки и её компоненты

Visual Studio Code – редактор исходного кода с возможностью расширения функциональности. Для разработки на Go используется расширение Go, которое обеспечивает подсветку синтаксиса, автодополнение, диагностику ошибок, форматирование кода, запуск тестов и отладку.

Основными элементами среды являются проводник файлов Explorer, редактор кода, панель Problems для просмотра ошибок и предупреждений, терминал Integrated Terminal, панель Output и боковая панель Extensions для установки расширений.

2.2 Основные приёмы работы с редактором кода

Редактор Visual Studio Code выполняет фоновый анализ кода: ошибки компиляции подчеркиваются, а комбинация Alt+Enter или Ctrl+. (Quick Actions) предлагает варианты исправления. Автодополнение IntelliSense (Ctrl+Space) подставляет имена классов и методов. Форматирование кода выполняется командой Ctrl+K, Ctrl+D. Навигация: F12 – переход к объявлению, Shift+F12 – поиск использований.

2.3 Сборка, запуск и отладка проекта

Сборка всех пакетов модуля выполняется командой `go build ./...`, запуск приложения – командой `go run ./cmd/weight-calculator`, запуск тестов – `go test ./...`. Команды выполняются во встроенном терминале Visual Studio Code. Для отладки используется расширение Go и отладчик Delve. Точка останова устанавливается щелчком слева от номера строки. Запуск отладки выполняется клавишей F5; пошаговое выполнение – F10, вход в вызываемую функцию – F11. Значения переменных отображаются в панели Variables.

2.4 Паттерн MVC

Паттерн MVC (Model-View-Controller) разделяет моделирование предметной области, визуальное отображение и действия пользователя на три компонента. Модель (Model) управляет поведением и данными приложения, реагируя на инструкции по изменению состояния. Представление (View) управляет выводом информации. Контроллер (Controller) интерпретирует ввод пользователя, сообщая модели или представлению о необходимости изменений.

2.5 Активная и пассивная модель

Паттерн MVC предполагает два способа обновления данных. В пассивной модели контроллер эксклюзивно управляет моделью: он вносит изменения, а затем напрямую сообщает представлению о необходимости обновления. Активная модель опирается на паттерн «Наблюдатель» (Observer): модель самостоятельно рассылает уведомления при смене своего состояния. Представление подписывается на эти оповещения и само запрашивает актуальные данные, тогда как контроллер лишь обрабатывает действия пользователя. В данной лабораторной работе активная модель реализована посредством уведомления `ModelUpdated` модели `WeightModel`.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Вариант 4 требует разработать программу «Калькулятор веса на разных планетах». Пользователь вводит вес человека на Земле. Программа рассчитывает и отображает вес на Меркурии, Венере, Земле, Марсе, Юпитере, Сатурне, Уране и Нептуне с учётом относительной силы тяжести. Ввод выполняется в отдельном окне по кнопке «Ввести данные»; последние корректные данные восстанавливаются при повторном открытии окна, а ошибки сопровождаются сообщением.

Среда разработки – Visual Studio Code, язык – Go 1.26.1, графическая библиотека – Fyne v2.6.3, операционная система – Ubuntu.

3.2 Настройка среды и расширений для UML и SysML v2

Установлена среда Microsoft Visual Studio Code. Создан go модуль с названием weight-calculator. В файле указана целевая версия go 1.26.1.

Для работы с SysML v2 и PlantUML можно использовать расширения для Visual Studio Code (например, PlantUML Integration) для генерации диаграмм. Структурная модель системы в текстовой нотации SysML v2 приведена на рисунке 3.1.

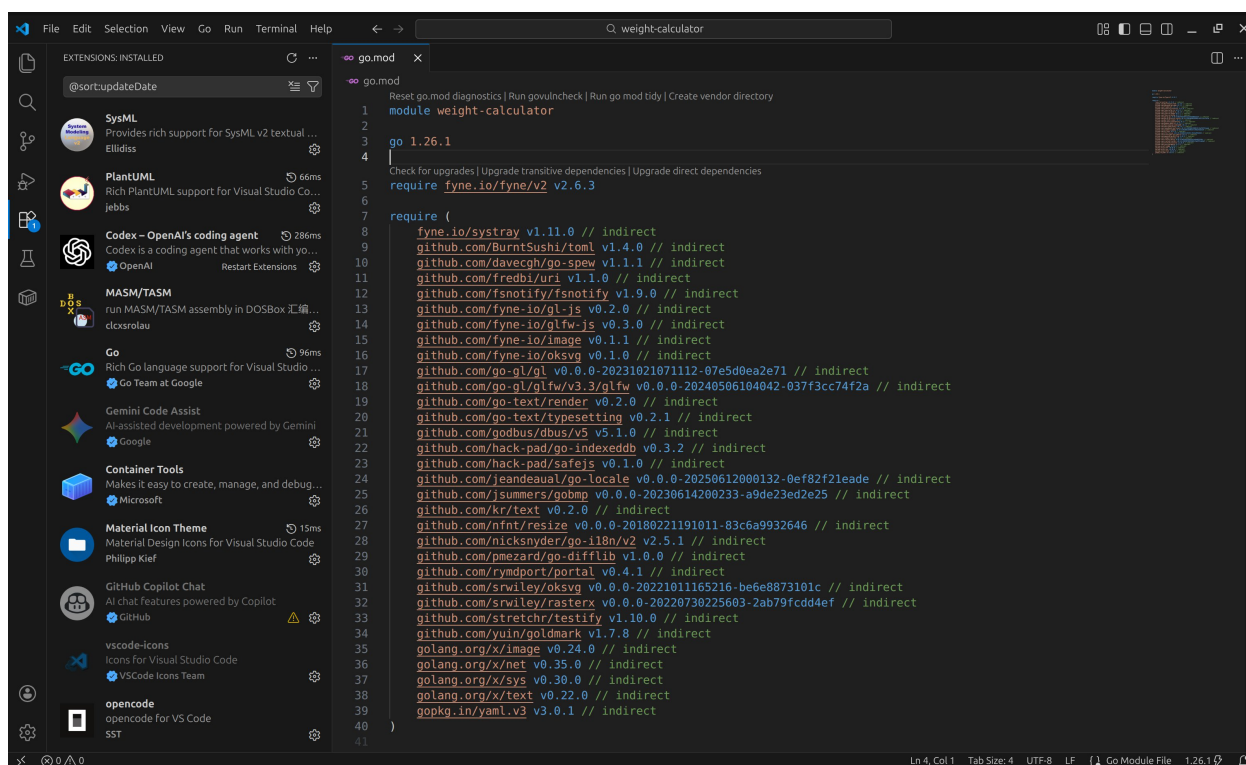


Рисунок 3.1 – Расширения Go, PlantUML и SysML v2 в Visual Studio Code

Структурное представление системы в нотации, близкой к SysML v2:

```

package WeightCalculator {
  part def WeightModel {
    attribute input : WeightInput;
    attribute result : WeightResult;
    port notify;
  }
  part def MainView {
    port listen;
    port userAction;
  }
  part def FyneInputDialog;
  part def WeightController {
    port handle;
  }
  part system {
    part model : WeightModel;
    part view : MainView;
    part dialog : FyneInputDialog;
    part controller : WeightController;
    connect view.userAction to controller.handle;
    connect model.notify to view.listen;
  }
}

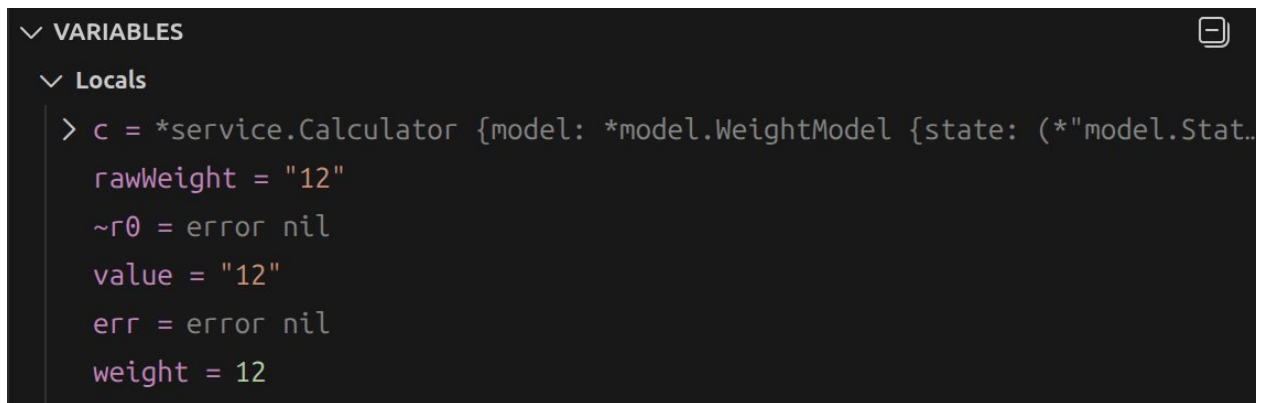
```

3.3 Демонстрация отладчика

Работа отладчика продемонстрирована на методе Calculate сервиса Calculator. Точка останова установлена в начале метода перед выполнением проверки введённого значения. После запуска приложения в режиме отладки и ввода значения 12 программа приостанавливается на установленной точке.

В окне Locals отображаются параметры и промежуточные результаты вычисления: исходная строка rawWeight = "12", нормализованное значение value = "12" и числовой результат weight = 12. Переменные err имеют значение nil, что означает отсутствие ошибок. Одна переменная относится к преобразованию строки в число, а другая – к результату вызова контроллера.

Пошаговое выполнение с помощью команды Step Over (F10) позволяет проследить проверку корректности ввода, замену запятой на точку, преобразование строки в число, расчёт веса на планетах и сохранение результата в модели.



```
✓ VARIABLES
  ✓ Locals
    > c = *service.Calculator {model: *model.WeightModel {state: (*"model.Stat..
      rawWeight = "12"
      ~r0 = error nil
      value = "12"
      err = error nil
      weight = 12
```

Рисунок 3.2 – Остановка программы в отладчике Visual Studio Code

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на пакеты, соответствующие компонентам MVC:

- model: активная модель WeightModel, структуры Planet, PlanetWeight и State для хранения состояния приложения;
- service: Calculator, выполняющий проверку введённого веса и расчёт значений для планет;
- controller: WeightController, связывающий действие пользователя и сервис;
- view: представления MainWindow и окно ввода веса, созданные средствами Fyne;
- cmd/weight-calculator: точка входа приложения.

В main.go создаются модель, калькулятор и контроллер, после чего вызывается view.Run. Главное окно подписывается на модель. Представление передаёт введённую строку контроллеру, контроллер вызывает Calculator. После успешной проверки Calculator формирует новое состояние и передаёт его модели. WeightModel сохраняет состояние и уведомляет представление методом ModelUpdated, после чего на главной форме обновляются результаты.

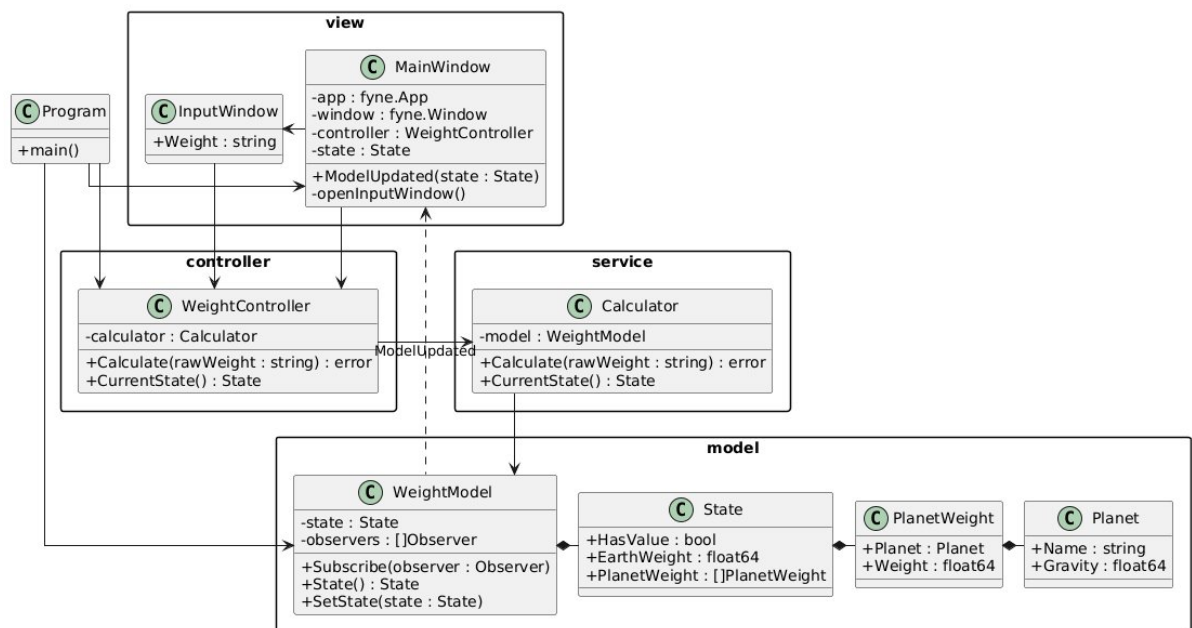


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

Модель реализована в пакете `internal/model`. Структура `Planet` содержит название планеты и коэффициент силы тяжести, `PlanetWeight` – планету и рассчитанный вес, а `State` – исходный вес и результаты расчёта. `WeightModel` хранит состояние и список наблюдателей: метод `Subscribe` добавляет представление, а `SetState` сохраняет новое состояние и вызывает `ModelUpdated` у всех подписчиков. Функция `Planets` задаёт коэффициенты для восьми планет. Модель не зависит от `Fyne`, поэтому логику хранения и расчёта можно проверять отдельными тестами `Go`.

3.6 Представление

Представление реализовано в пакете `internal/view` структурой `MainWindow`. Главное окно содержит кнопку ввода, метку веса на Земле и область вывода результатов. Приложение создаётся с постоянным идентификатором, поэтому настройки `Fyne` доступны после повторного запуска. При создании главного окна из настроек считывается последний корректный вес; при наличии значения оно подставляется в поле при открытии формы ввода. После подтверждения исходная строка передаётся контроллеру, а ошибка выводится диалогом `Fyne`. `MainWindow` реализует интерфейс `Observer`: метод `ModelUpdated` обновляет метку и список рассчитанных значений. Представление не выполняет расчёт и проверку данных.

3.7 Контроллер

Контроллер `WeightController` из пакета `internal/controller` хранит ссылку на `Calculator` и предоставляет методы `Calculate` и `CurrentState`. Метод `Calculate` получает строку из формы и передаёт её сервису, который проверяет значение, рассчитывает вес и изменяет состояние модели. При ошибке она возвращается в представление для вывода пользователю. После успешного расчёта представление получает текущий результат методом `CurrentState` и сохраняет вес в настройках `Fyne`. Таким образом, контроллер принимает действие пользователя и вызывает бизнес-логику.

3.8 Запоминание данных и обработка ошибок

После успешной проверки и расчёта вес на Земле сохраняется в `State` модели и в настройках `Fyne` методом `SetFloat`. При следующем запуске приложения значение считывается методом `FloatWithFallback` и подставляется в поле ввода. Обработка ошибок выполняется сервисом `Calculator` до изменения модели и сохранения настроек. Проверяются пустая строка, отрицательное или нулевое значение, буквы, несколько десятичных разделителей, невозможность преобразования, `NaN` и бесконечность. Запятая допускается как десятичный разделитель и заменяется на точку перед преобразованием в число. При ошибке `SetState` не вызывается, поэтому предыдущие результаты и сохранённое значение не изменяются. При корректном вводе для каждой планеты применяется формула $W_{\text{планеты}} = W_{\text{Земли}} \times G_{\text{планеты}}$, а новые данные передаются модели и отображаются в окне.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В результате выполнения лабораторной работы в среде Visual Studio Code разработано приложение на языке Go с графическим интерфейсом Fyne для расчёта веса человека на планетах Солнечной системы. Приложение построено по паттерну MVC с активной моделью. Проверены ввод положительного веса, расчёт значений для всех планет, обработка пустой строки, буквенных, нулевых и отрицательных значений, а также подстановка последнего корректного веса при повторном открытии окна ввода. Экранные формы приведены на рисунках 4.1–4.4, исходный код программы – в приложении А.

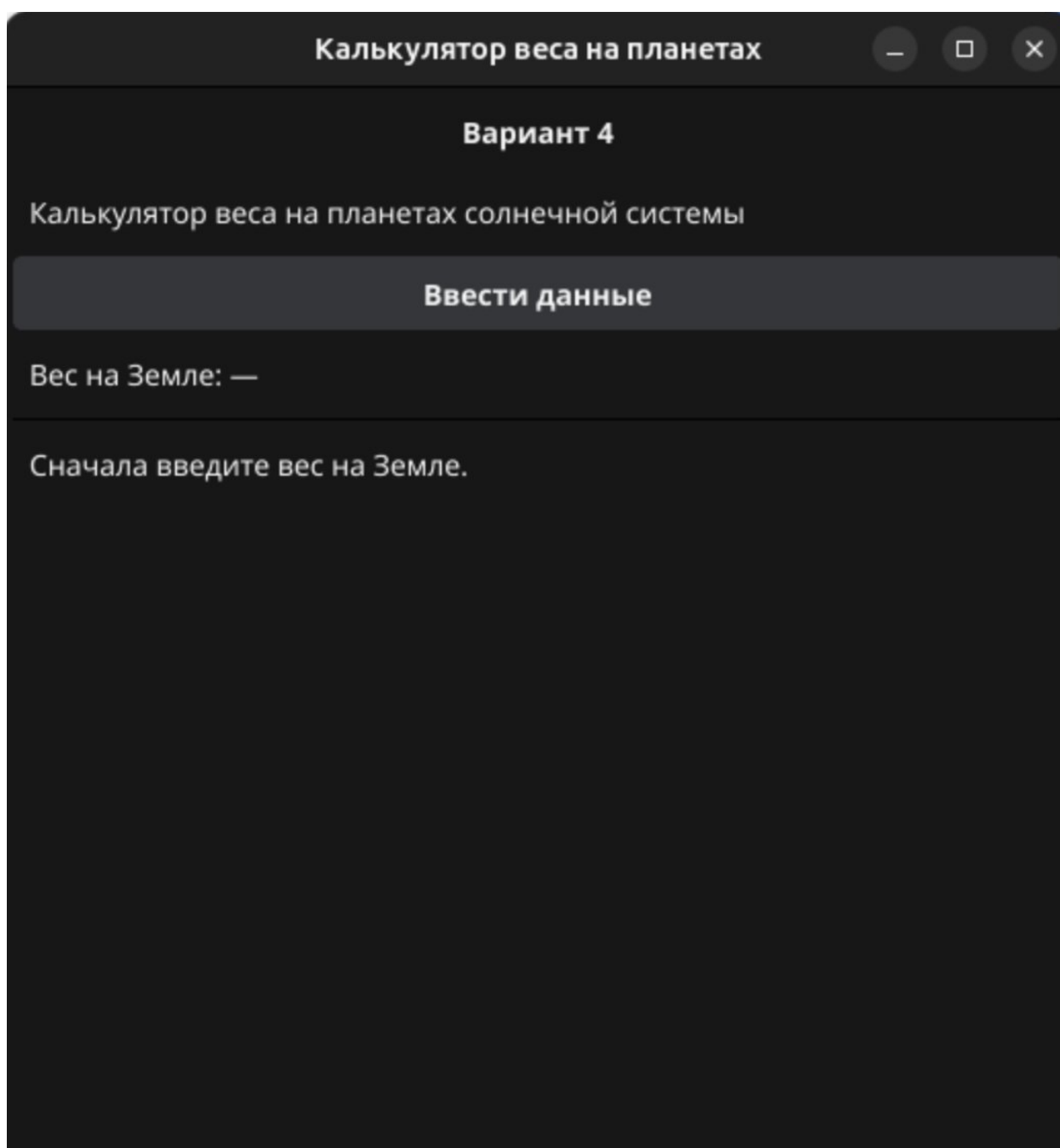


Рисунок 4.1 – Главная форма после запуска

Ввод веса

Введите вес человека на Земле (кг):

23.00

Рассчитать Отмена

Рисунок 4.2 – Окно ввода данных

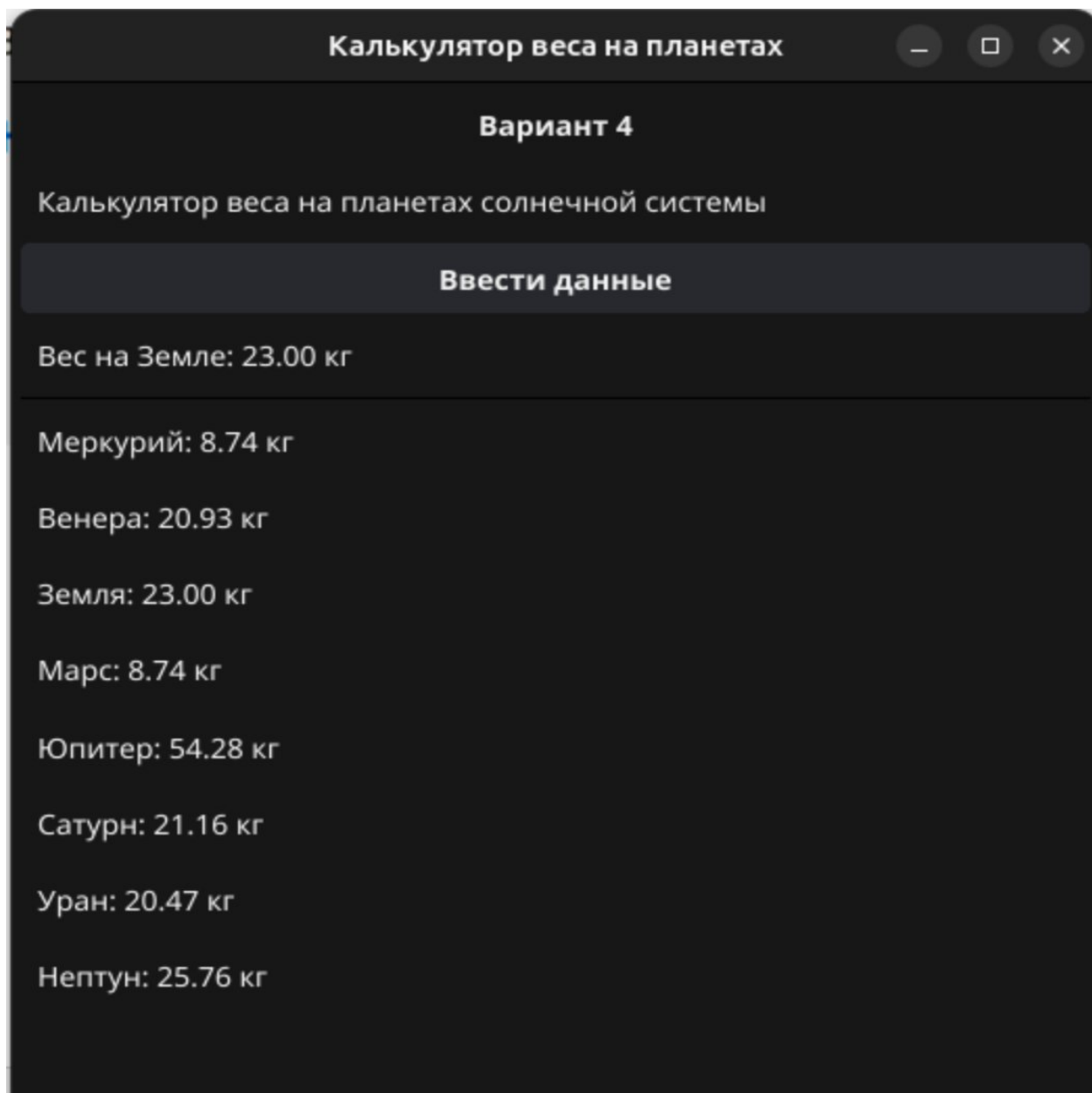


Рисунок 4.3 – Результат вычисления

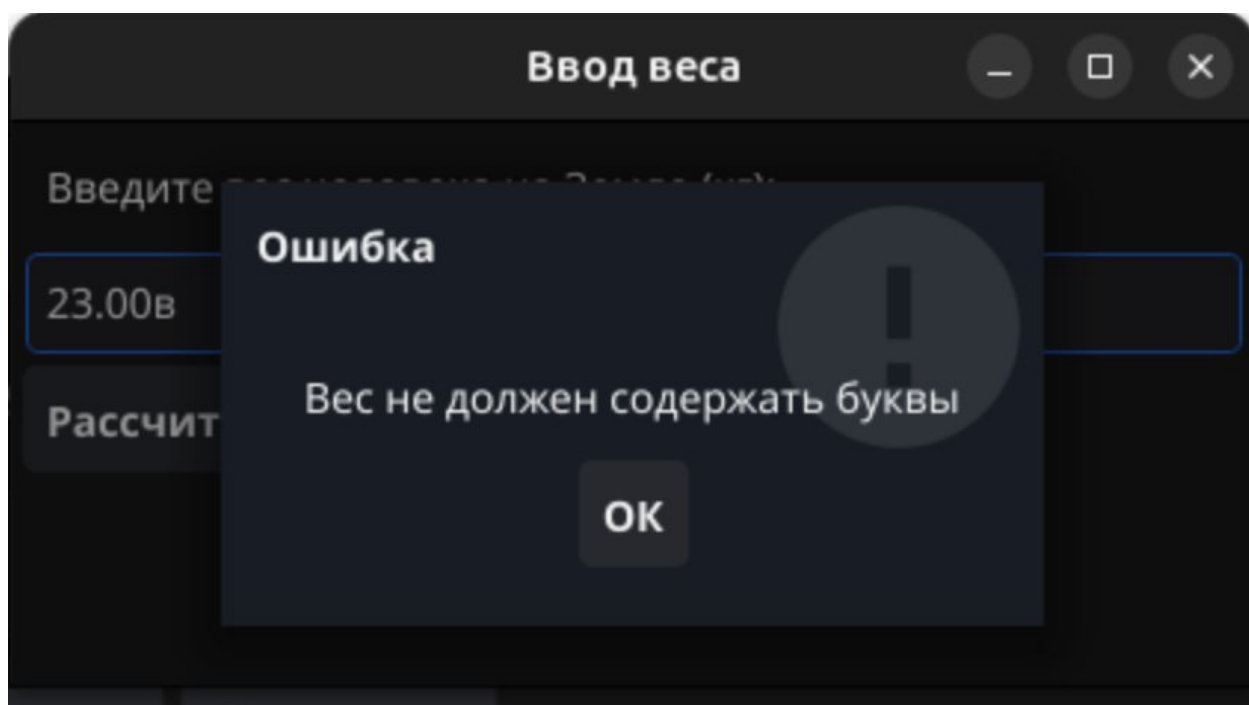


Рисунок 4.4 – Сообщение при некорректном значении веса

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор или интерпретатор, система сборки, отладчик, средства запуска, навигация и рефакторинг, интеграция с системой контроля версий, терминал и расширения. В Visual Studio Code эти функции представлены Explorer, Search, Source Control, Run and Debug, Extensions, Problems и Terminal; языковые функции Go предоставляет gopls.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

Основные приёмы включают автодополнение, быстрые исправления, переименование, поиск определений и ссылок, форматирование gofmt, организацию импортов, сворачивание блоков и сравнение версий Git. Расширение Go выполняет фоновую диагностику и отмечает часть ошибок до сборки.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

Команда `go build ./...` компилирует пакеты модуля, `go run ./cmd/weight-calculator` собирает и запускает приложение, `go test ./...` выполняет тесты. Эти команды запускаются в терминале Visual Studio Code. Для пошаговой отладки используется расширение Go и отладчик Delve.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, разделяющий данные, графический интерфейс и обработку действий пользователя. Model хранит состояние приложения и уведомляет подписчиков об его изменении, View отображает данные, Controller принимает действие пользователя и передаёт его бизнес-логике. В данной работе роль модели выполняет WeightModel, представления – MainWindow и окно ввода, контроллера – WeightController, а расчёт и проверку данных выполняет сервис Calculator.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление: после изменения данных контроллер отдельно иницирует обновление View. Активная модель реализует подписку и после изменения сама вызывает наблюдателей. В работе WeightModel хранит список Observer и вызывает ModelUpdated, а MainWindow получает новое состояние и обновляет элементы интерфейса.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные компоненты Visual Studio Code, расширение Go, форматирование gofmt, сборка, тестирование и отладка с Delve. Для представления архитектуры приложения подготовлена диаграмма типов и интерфейсов.

Разработана программа расчёта веса человека на планетах Солнечной системы на языке Go с интерфейсом Fyne. Она принимает вес на Земле и отображает рассчитанные значения для восьми планет. Последнее корректное значение сохраняется в настройках Fyne и подставляется при повторном открытии окна ввода, в том числе после перезапуска приложения.; пустые, нечисловые, нулевые и отрицательные значения сопровождаются сообщениями об ошибке. WeightModel хранит состояние и уведомляет MainWindow, а WeightController передаёт действия пользователя сервису расчёта.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

Файл cmd/weight-calculator/main.go

```
package main

import (
    "weight-calculator/internal/controller"
    "weight-calculator/internal/model"
    "weight-calculator/internal/service"
    "weight-calculator/internal/view"
)

func main() {
    weightModel := model.NewWeightModel()
    calculator := service.NewCalculator(weightModel)
    weightController := controller.NewWeightController(calculator)

    view.Run(weightModel, weightController)
}
```

Файл internal/controller/controller.go

```
package controller

import (
    "weight-calculator/internal/model"
    "weight-calculator/internal/service"
)

// WeightController handles user actions and delegates business work to the
// service.
type WeightController struct {
    calculator *service.Calculator
}

func NewWeightController(calculator *service.Calculator) *WeightController {
    return &WeightController{calculator: calculator}
}

func (c *WeightController) Calculate(rawWeight string) error {
    return c.calculator.Calculate(rawWeight)
}
```

```

}

func (c *WeightController) CurrentState() model.State {
    return c.calculator.CurrentState()
}

```

Файл `internal/model/model.go`

```

package model

// Planet describes a planet and its gravity relative to Earth's gravity.
type Planet struct {
    Name      string
    Gravity   float64
}

// PlanetWeight is a calculated weight for one planet.
type PlanetWeight struct {
    Planet Planet
    Weight float64
}

// State is the data displayed by the view.
type State struct {
    HasValue      bool
    EarthWeight   float64
    PlanetWeight []PlanetWeight
}

// Observer receives a notification when the model changes.
type Observer interface {
    ModelUpdated(State)
}

// WeightModel stores the current calculator state and notifies its
// observers.
// The notification is what makes this an active model in MVC.
type WeightModel struct {
    state      State
    observers []Observer
}

func NewWeightModel() *WeightModel {
    return &WeightModel{}
}

```

```

}

func (m *WeightModel) Subscribe(observer Observer) {
    m.observers = append(m.observers, observer)
}

func (m *WeightModel) State() State {
    return copyState(m.state)
}

// SetState changes the model and actively informs all subscribed views.
func (m *WeightModel) SetState(state State) {
    m.state = copyState(state)
    for _, observer := range m.observers {
        observer.ModelUpdated(copyState(m.state))
    }
}

func copyState(state State) State {
    state.PlanetWeight = append([]PlanetWeight(nil), state.PlanetWeight...)
    return state
}

// Planets returns the gravity constants used by the calculator.
func Planets() []Planet {
    return []Planet{
        {Name: "Меркурий", Gravity: 0.38},
        {Name: "Венера", Gravity: 0.91},
        {Name: "Земля", Gravity: 1.00},
        {Name: "Марс", Gravity: 0.38},
        {Name: "Юпитер", Gravity: 2.36},
        {Name: "Сатурн", Gravity: 0.92},
        {Name: "Уран", Gravity: 0.89},
        {Name: "Нептун", Gravity: 1.12},
    }
}

```

Файл `internal/service/calculator.go`

```

package service

import (
    "errors"
    "math"
    "strconv"

```

```

"strings"
"unicode"

"weight-calculator/internal/model"
)

var (
    errEmptyWeight          = errors.New("введите вес")
    errNegativeWeight       = errors.New("вес не может быть отрицательным")
    errMultipleSeparators  = errors.New("в числе может быть не более одной
точки или запятой")
    errLettersInWeight     = errors.New("вес не должен содержать буквы")
    errInvalidWeight       = errors.New("введите положительное число,
например 70")
    errNonPositiveWeight   = errors.New("вес должен быть больше нуля")
)

// Calculator contains the application's business logic.
type Calculator struct {
    model *model.WeightModel
}

func NewCalculator(weightModel *model.WeightModel) *Calculator {
    return &Calculator{model: weightModel}
}

// Calculate validates the text from the input field and updates the model.
func (c *Calculator) Calculate(rawWeight string) error {
    value := strings.TrimSpace(rawWeight)
    if value == "" {
        return errEmptyWeight
    }
    if strings.HasPrefix(value, "-") {
        return errNegativeWeight
    }
    if strings.Count(value, ".")+strings.Count(value, ",") > 1 {
        return errMultipleSeparators
    }
    if strings.IndexFunc(value, unicode.IsLetter) >= 0 {
        return errLettersInWeight
    }

    value = strings.ReplaceAll(value, ",", ".")
    weight, err := strconv.ParseFloat(value, 64)
    if err != nil || math.IsNaN(weight) || math.IsInf(weight, 0) {
        return errInvalidWeight
    }
}

```

```

    }
    if weight < 0 {
        return errNegativeWeight
    }
    if weight == 0 {
        return errNonPositiveWeight
    }

    planetWeights := make([]model.PlanetWeight, 0, len(model.Planets()))
    for _, planet := range model.Planets() {
        planetWeights = append(planetWeights, model.PlanetWeight{
            Planet: planet,
            Weight: weight * planet.Gravity,
        })
    }

    c.model.SetState(model.State{
        HasValue:    true,
        EarthWeight: weight,
        PlanetWeight: planetWeights,
    })
    return nil
}

func (c *Calculator) CurrentState() model.State {
    return c.model.State()
}

```

Файл `internal/view/format.go`

```

package view

import "strconv"

func formatWeight(weight float64) string {
    return strconv.FormatFloat(weight, 'f', 2, 64)
}

```

Файл `internal/view/input_window.go`

```

package view

import (
    "fyne.io/fyne/v2"

```

```

    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/dialog"
    "fyne.io/fyne/v2/widget"

    "weight-calculator/internal/controller"
    "weight-calculator/internal/model"
)

func newInputWindow(
    a fyne.App,
    weightController *controller.WeightController,
    state model.State,
) {
    inputWindow := a.NewWindow("Ввод веса")
    inputWindow.Resize(fyne.NewSize(420, 190))

    weightEntry := widget.NewEntry()
    weightEntry.SetPlaceholder("Например, 70")
    if state.HasValue {
        weightEntry.SetText(formatWeight(state.EarthWeight))
    }

    calculateButton := widget.NewButton("Рассчитать", func() {
        if err := weightController.Calculate(weightEntry.Text); err !=
nil {
            dialog.ShowError(err, inputWindow)
            return
        }
        a.Preferences().SetFloat(lastEarthWeightKey,
weightController.CurrentState().EarthWeight)
        inputWindow.Close()
    })

    cancelButton := widget.NewButton("Отмена", inputWindow.Close)
    inputWindow.SetContent(container.NewVBox(
        widget.NewLabel("Введите вес человека на Земле (кг):"),
        weightEntry,
        container.NewHBox(calculateButton, cancelButton),
    ))
    inputWindow.Show()
}

```

Файл `internal/view/main_window.go`

```
package view
```

```

import (
    "fmt"
    "math"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/app"
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/widget"

    "weight-calculator/internal/controller"
    "weight-calculator/internal/model"
)

const (
    applicationID      = "by.bsuir.weight-calculator"
    lastEarthWeightKey = "last_earth_weight"
)

// MainWindow is the main screen and an observer of the active model.
type MainWindow struct {
    app      fyne.App
    window   fyne.Window
    controller *controller.WeightController

    earthWeightLabel *widget.Label
    results          *fyne.Container
    state            model.State
}

func newMainWindow(weightModel *model.WeightModel, weightController
*controller.WeightController) *MainWindow {
    mainView := &MainWindow{
        app:      app.NewWithID(applicationID),
        controller: weightController,
        earthWeightLabel: widget.NewLabel("Вес на Земле: -"),
        results:      container.NewVBox(widget.NewLabel("Сначала
введите вес на Земле.")),
    }
    mainView.window = mainView.app.NewWindow("Калькулятор веса на
планетах")
    weightModel.Subscribe(mainView)
    mainView.restoreLastWeight()

    mainView.window.Resize(fyne.NewSize(520, 520))
    mainView.window.SetContent(container.NewVBox(

```

```

        widget.NewLabelWithStyle("Вариант 4", fyne.TextAlignCenter,
fyne.TextStyle{Bold: true}),
        widget.NewLabel("Калькулятор веса на планетах солнечной
системы"),
        widget.NewButton("Ввести данные", mainView.openInputWindow),
        mainView.earthWeightLabel,
        widget.NewSeparator(),
        mainView.results,
    ))
    return mainView
}

// restoreLastWeight loads the last valid value saved by Fyne preferences.
func (v *MainWindow) restoreLastWeight() {
    weight := v.app.Preferences().FloatWithFallback(lastEarthWeightKey, 0)
    if weight > 0 && !math.IsNaN(weight) && !math.IsInf(weight, 0) {
        v.state = model.State{HasValue: true, EarthWeight: weight}
    }
}

// ModelUpdated is called by WeightModel after a successful calculation.
func (v *MainWindow) ModelUpdated(state model.State) {
    v.state = state
    if !state.HasValue {
        return
    }

    v.earthWeightLabel.SetText(fmt.Sprintf("Вес на Земле: %.2f кг",
state.EarthWeight))
    v.results.RemoveAll()
    for _, result := range state.PlanetWeight {
        v.results.Add(widget.NewLabel(fmt.Sprintf("%s: %.2f кг",
result.Planet.Name, result.Weight)))
    }
    v.results.Refresh()
}

func (v *MainWindow) openInputWindow() {
    newInputWindow(v.app, v.controller, v.state)
}

// Run creates and displays the application window.
func Run(weightModel *model.WeightModel, weightController
*controller.WeightController) {
    mainView := newMainWindow(weightModel, weightController)
    mainView.window.ShowAndRun()
}

```